



Solana Private Channels Offchain

Security Assessment

Prepared for
Solana Foundation

Akash Gurugunti
Gabriel Ottoboni

Prepared by
Otter Audits, LLC

Prepared on
September 2nd, 2026

sudOu53r.ak@osec.io
ottoboni@osec.io

Table of Contents

1. Executive Summary	3
1.1. Overview	3
1.2. Key Findings	3
2. Scope	4
3. Findings	5
3.1. Severity by component	5
3.2. Findings summary	5
4. Advisories	6
OS-SPF-ADV-00	● Withdrawal Recovery Can Remint Deposits 8
OS-SPF-ADV-01	● Data-less Account Capping Deletes Nonce Tombstones 10
OS-SPF-ADV-02	● Stale-Tree Withdrawals Are Retried Indefinitely 12
OS-SPF-ADV-03	● InitializeMint2 Missing From Admin Instruction Map 14
OS-SPF-ADV-04	● Excess Token Transfers Block Reconciliation 15
OS-SPF-ADV-05	● Insufficient Finality for Operator Transactions 16
OS-SPF-ADV-06	● JIT Mint Retry Bypasses Signature Persistence 17
OS-SPF-ADV-07	● Database-Loaded Accounts Never Become Evictable 19
OS-SPF-ADV-08	● Admin VM Returns Misaligned Loaded Accounts 21
OS-SPF-ADV-09	● Failed Transaction Account Updates 22
OS-SPF-ADV-10	● Zero-Lamport Accounts Retain Data in DB 23
OS-SPF-ADV-11	● Database Errors Treated as Missing Accounts 24
OS-SPF-ADV-12	● Failed Withdrawal Remint Sent to Destination 26
OS-SPF-ADV-13	● Panic on Duplicate Transaction Account Keys 27
OS-SPF-ADV-14	● Overly Broad Withdrawal Pipeline Quarantine 28
OS-SPF-ADV-15	● Non-Durable Tree Rotation Dispatch 30
OS-SPF-ADV-16	● Backpressure-Induced Duplicate Deposit Mints 33
OS-SPF-ADV-17	● Cross-Role Pending Remint Recovery 35
OS-SPF-ADV-18	● Pre-Broadcast Persistence Failure Leaves Phantom SMT Leaf 37
OS-SPF-ADV-19	● Startup SMT Validation Ignores Recoverable Withdrawals 39
OS-SPF-ADV-20	● Pre-Broadcast SMT Failures Quarantined as Withdrawals 41
OS-SPF-ADV-21	● Checkpoint Leapfrogging Falsely Proves Release Absence 43
OS-SPF-ADV-22	● Fallible Boundary Withdrawal Blocks Bitmap Rotation 45
OS-SPF-ADV-23	● Withdrawal Builder Errors Skip Remint 48

<u>OS-SPF-ADV-24</u>	● Unbounded Recursive ResetSmtRoot Confirmation Re-tries	49
<u>OS-SPF-ADV-25</u>	● Missing Source RPC Requirement for Escrow Operator	51
<u>OS-SPF-ADV-26</u>	● Fee Payer Monitor Tracks Operator Wallet	55
<u>OS-SPF-ADV-27</u>	● Nonce-Keyed Cleanup Removes Unrelated Mint Builder	57
<u>OS-SPF-ADV-28</u>	● Token-2022 Accounts Abort Escrow Reconciliation	59
<u>OS-SPF-ADV-29</u>	● Unscoped CPI Error Decoding Can Stall Bitmap Rotations	61
5. Suggestions		63
<u>OS-SPF-SUG-00</u>	● Single-Byte Admin Instruction Identifiers	64
<u>OS-SPF-SUG-01</u>	● Fetcher Poll Sleep Delays Shutdown	65
<u>OS-SPF-SUG-02</u>	● Permanent Suppression of Orphan Deposit Alerts	67
A. Vulnerability Rating Scale		68
B. Procedure		69

1. Executive Summary

1.1. Overview

Solana Foundation engaged OtterSec to conduct a security assessment of the `offchain` program within Solana Private Channels. This assessment was conducted between June 25th and July 20th, 2026. For information on our auditing methodology, refer to [Appendix B](#).

1.2. Key Findings

We produced 33 findings throughout this audit engagement.

Among the most impactful findings, we identified that withdrawal recovery can remint deposits when stale Processing rows are recovered across operator boundaries and the wrong RPC client is used, which can create unbacked tokens without corresponding escrow backing (● [OS-SPF-ADV-00](#)). We also found that capping writable zero-byte accounts to zero lamports deletes nonce tombstones, undermining the persistent marker the DvP flow relies on for replay protection and allowing nonce reuse (● [OS-SPF-ADV-01](#)).

In the withdrawal pipeline, stale-tree rows can be retried indefinitely, so a single bad release can keep the tree stuck and block later withdrawals from making progress (● [OS-SPF-ADV-02](#)). On the deposit side, backpressure can leave rows marked Processing before a durable mint signature exists, allowing recovery to requeue the same deposit and mint it twice (● [OS-SPF-ADV-16](#)). Separately, a pre-broadcast SMT persistence failure can leave a phantom leaf in local state, causing later withdrawal proofs to fail until the sender is reinitialized (● [OS-SPF-ADV-18](#)).

Finally, startup SMT validation only considers completed withdrawals, so a landed release that is still recorded as `pending_remint` or `manual_review` can make the sender refuse to start even though the on-chain tree is already correct (● [OS-SPF-ADV-19](#)).

2. Scope

`core` is the on-channel runtime that processes private-channel transactions through dedup, signature verification, sequencing, execution, and settlement. It enforces the channel's transaction rules and persists the resulting state.

`indexer` is the off-chain service that watches Solana Mainnet and the private channel for deposits and withdrawals, records them in the database, and drives follow-up actions. It keeps the channel and escrow state synchronized through its operator pipeline.

The source code was delivered to us in a Git repository at

<https://github.com/solana-foundation/solana-private-channels>. This audit was performed against commits [977327b](#) and [d2a25b3](#).

3. Findings

Findings were rated using the criteria in [Appendix A](#).

We split the findings into **advisories** and **suggestions**. Advisories have an immediate impact and should be remediated as soon as possible. Suggestions do not have an immediate impact but will aid in mitigating future vulnerabilities.

Items are tracked with a stable identifier: **OS-SPF-ADV-*** for advisories and **OS-SPF-SUG-*** for suggestions.

3.1. Severity by component

Component	● Critical	● High	● Medium	● Low	● Info	Total
Indexer	1	1	13	7	2	24
Core	—	1	7	—	1	9
Total	1	2	20	7	3	33

3.2. Findings summary

Severity	Count	
● Critical	1	
● High	2	
● Medium	20	
● Low	7	
● Informational	3	

4. Advisories

Here, we present a technical analysis of the 30 vulnerabilities we identified during our audit. These vulnerabilities have **immediate** security implications, and we recommend remediation as soon as possible.

ID	Severity	Status	Description
OS-SPF-ADV-00	● Critical (Indexer)	✓ Resolved	Withdrawal Recovery Can Remint Deposits
OS-SPF-ADV-01	● High (Core)	✓ Resolved	Data-less Account Capping Deletes Nonce Tombstones
OS-SPF-ADV-02	● High (Indexer)	✓ Resolved	Stale-Tree Withdrawals Are Retried Indefinitely
OS-SPF-ADV-03	● Medium (Core)	✓ Resolved	InitializeMint2 Missing From Admin Instruction Map
OS-SPF-ADV-04	● Medium (Indexer)	✓ Resolved	Excess Token Transfers Block Reconciliation
OS-SPF-ADV-05	● Medium (Indexer)	✓ Resolved	Insufficient Finality for Operator Transactions
OS-SPF-ADV-06	● Medium (Indexer)	✓ Resolved	JIT Mint Retry Bypasses Signature Persistence
OS-SPF-ADV-07	● Medium (Core)	✓ Resolved	Database-Loaded Accounts Never Become Evictable
OS-SPF-ADV-08	● Medium (Core)	✓ Resolved	Admin VM Returns Misaligned Loaded Accounts
OS-SPF-ADV-09	● Medium (Core)	✓ Resolved	Failed Transaction Account Updates
OS-SPF-ADV-10	● Medium (Core)	✓ Resolved	Zero-Lamport Accounts Retain Data in DB
OS-SPF-ADV-11	● Medium (Core)	✓ Resolved	Database Errors Treated as Missing Accounts
OS-SPF-ADV-12	● Medium (Indexer)	✓ Resolved	Failed Withdrawal Remint Sent to Destination
OS-SPF-ADV-13	● Medium (Core)	✓ Resolved	Panic on Duplicate Transaction Account Keys
OS-SPF-ADV-14	● Medium (Indexer)	✓ Resolved	Overly Broad Withdrawal Pipeline Quarantine
OS-SPF-ADV-15	● Medium (Indexer)	✓ Resolved	Non-Durable Tree Rotation Dispatch

ID	Severity	Status	Description
OS-SPF-ADV-16	● Medium (Indexer)	✓ Resolved	Backpressure-Induced Duplicate Deposit Mints
OS-SPF-ADV-17	● Medium (Indexer)	✓ Resolved	Cross-Role Pending Remint Recovery
OS-SPF-ADV-18	● Medium (Indexer)	✓ Resolved	Pre-Broadcast Persistence Failure Leaves Phantom SMT Leaf
OS-SPF-ADV-19	● Medium (Indexer)	✓ Resolved	Startup SMT Validation Ignores Recoverable Withdrawals
OS-SPF-ADV-20	● Medium (Indexer)	✓ Resolved	Pre-Broadcast SMT Failures Quarantined as Withdrawals
OS-SPF-ADV-21	● Medium (Indexer)	✓ Resolved	Checkpoint Leapfrogging Falsely Proves Release Absence
OS-SPF-ADV-22	● Medium (Indexer)	✓ Resolved	Fallible Boundary Withdrawal Blocks Bitmap Rotation
OS-SPF-ADV-23	● Low (Indexer)	○ Ack'd	Withdrawal Builder Errors Skip Remint
OS-SPF-ADV-24	● Low (Indexer)	○ Ack'd	Unbounded Recursive ResetSmtRoot Confirmation Retries
OS-SPF-ADV-25	● Low (Indexer)	○ Ack'd	Missing Source RPC Requirement for Escrow Operator
OS-SPF-ADV-26	● Low (Indexer)	○ Ack'd	Fee Payer Monitor Tracks Operator Wallet
OS-SPF-ADV-27	● Low (Indexer)	○ Ack'd	Nonce-Keyed Cleanup Removes Unrelated Mint Builder
OS-SPF-ADV-28	● Low (Indexer)	○ Ack'd	Token-2022 Accounts Abort Escrow Reconciliation
OS-SPF-ADV-29	● Low (Indexer)	○ Ack'd	Unscoped CPI Error Decoding Can Stall Bitmap Rotations

Withdrawal Recovery Can Remint Deposits

ID	Severity	Status
OS-SPF-ADV-00	● Critical	✓ Resolved

Description

The stale-processing query selects every `processing` transaction whose `updated_at` is older than the supplied threshold, without restricting the result set by operator `program_type` or transaction direction. Consequently, operators sharing the transaction database can load each other's in-flight rows during recovery, as shown in `get_stale_processing_transactions_internal`.

```
indexer/src/storage/postgres/db.rs
```

```

1  pub async fn get_stale_processing_transactions_internal(
2      &self,
3      threshold: Duration,
4      limit: i64,
5  ) -> Result<Vec<DbTransaction>, sqlx::Error> {
6      let threshold_secs = threshold.as_secs() as f64;
7      sqlx::query_as::<_, DbTransaction>(&format!(
8          r#"
9          SELECT
10             {}, {}, {}, {}, {}, {}, {}, {}, {}, {},
11             {}, {}, {}, {}, {}, {}, {}, {}, {}, {}, {}, {},
12          FROM transactions
13          WHERE {} = 'processing'
14              AND {} < NOW() - make_interval(secs => $1)
15          ORDER BY {} ASC
16          LIMIT $2
17          "#, [...])
18      ))
19      .bind(threshold_secs)
20      .bind(limit)
21      .fetch_all(&self.pool)
22      .await
23  }
```

Stale processing rows are selected without an operator-type filter.

`decide_action` determines recovery behavior solely from the row's `transaction_type` and the supplied RPC client. When a withdraw operator processes a deposit row, deposit signature finality is therefore evaluated through the withdraw operator's Solana RPC client rather than the private-channel chain on which the mint was submitted, as shown in `decide_action`.

indexer/src/operator/recovery.rs

```

1  async fn decide_action(
2      row: &DbTransaction,
3      storage: &Storage,
4      rpc_client: &RpcClientWithRetry,
5  ) -> RecoveryAction {
6      let action = match row.transaction_type {
7          TransactionType::Deposit => match check_deposit(row, storage, rpc_client).await
8              {
9              DepositOutcome::Landed { signature } => RecoveryAction::Complete
10                 { signature },
11              DepositOutcome::NotLanded => RecoveryAction::Demote,
12              DepositOutcome::Live { reason } => RecoveryAction::NoAction { reason },
13              DepositOutcome::Ambiguous { reason } => RecoveryAction::Quarantine
14                 { reason },
15          },
16          TransactionType::Withdrawal => match check_withdrawal(row, storage,
17              rpc_client).await {
18              WithdrawalAction::Complete { signature } => RecoveryAction::Complete
19                 { signature },
20              WithdrawalAction::Demote => RecoveryAction::Demote,
21              WithdrawalAction::LeaveProcessing { reason } => RecoveryAction::NoAction
22                 { reason },
23              WithdrawalAction::Quarantine { reason } => RecoveryAction::Quarantine
24                 { reason },
25          },
26      };[...]
```

Recovery classification receives no program-type or source-chain context.

A mint signature that is unknown to that RPC client (and where Solana's block height far exceeds the channel's stored `last_valid_block_height`) can be classified as dead and the deposit can be demoted from `Processing` to `Pending`. The demoted row is then re-minted unconditionally because the deposit path stamps `mint_idempotency_memo` but never reads it. This may result in an already-landed deposit mint to be reissued, creating unbacked-channel tokens without corresponding additional escrow backing on every withdraw-operator restart via the `Duration::ZERO` boot sweep.

Remediation

Scope stale-processing recovery to the operator's domain before rows are classified. In addition, pass and use the appropriate source-chain RPC client when verifying deposit mint signatures.

Patch

Resolved in [7f5a5c0](#).

Data-less Account Capping Deletes Nonce Tombstones

ID	Severity	Status
OS-SPF-ADV-01	● High	✓ Resolved

Description

The private-channel execution path reduces the lamport balance of every writable, data-less account to zero after execution. Consequently, a zero-byte account that relies on its existence and lamports for persistence is removed when its transaction is settled, unlike an account model that permits data-less accounts with lamports.

core/src/stages/execution.rs

```

402 fn cap_lamports(
403     output: &mut LoadAndExecuteSanitizedTransactionsOutput,
404     transactions: &[SanitizedTransaction],
405 ) {
406     for (result, tx) in output
407         .processing_results
408         .iter_mut()
409         .zip(transactions.iter())
410     {
411         let Ok(ProcessedTransaction::Executed(executed)) = result else {
412             continue;
413         };
414         for (index, (_, acct)) in
415             executed.loaded_transaction.accounts.iter_mut().enumerate() {
416             // Only writable accounts are persisted by the settler / BOB update.
417             if !tx.is_writable(index) {
418                 continue;
419             }
420             let cap = if acct.data().is_empty() { 0 } else { 1 };
421             if acct.lamports() > cap {
422                 acct.set_lamports(cap);
423             }
424         }
425     }

```

The DvP program's PDA creation helper funds a new account with at least one lamport, including when its requested allocation size is zero. This supplies lamports to a nonce tombstone but does not provide data that would survive the execution-stage cap. `process_create_dvp` creates the nonce tombstone as a zero-byte, program-owned PDA intended to mark nonce use. Because that account is writable during creation, the execution-stage cap removes its lamports after the instruction completes, defeating the persistent marker.

pre>
 dvp-swap-program/program/src/processor/create_dvp.rs

```

1  pub fn process_create_dvp(...) -> ProgramResult {
2      [...]
3      create_pda_account(
4          payer_info,
5          &rent,
6          0,
7          program_id,
8          nonce_tombstone_info,
9          tombstone_seeds,
10     );
11     [...]
12 }

```

Before creating a DvP, `process_create_dvp` treats a system-owned nonce tombstone as unused. Once the zero-byte tombstone is deleted, a later call using the same nonce can satisfy this check and recreate the DvP PDA. Thus, an attacker may front-run a deposit by reinitializing that PDA with a different settlement destination and steal the deposited funds.

pre>
 dvp-swap-program/program/src/processor/create_dvp.rs

```

1  pub fn process_create_dvp(...) -> ProgramResult {
2      [...]
3      );
4      require!(
5          nonce_tombstone_info.owned_by(&pinocchio_system::ID),
6          DvpSwapProgramError::NonceAlreadyUsed
7      );
8      [...]
9  }

```

Remediation

Update `cap_lamports` so it does not reduce writable zero-byte accounts to zero solely because they have no data.

Patch

Resolved in [PR#228](#).

Stale-Tree Withdrawals Are Retried Indefinitely

ID	Severity	Status
OS-SPF-ADV-02	● High	✓ Resolved

Description

A failed `ReleaseFunds` withdrawal can leave an unresolved ambiguous nonce for its SMT tree. As shown in `handle_transaction_submission`, subsequent withdrawals in that tree are parked rather than submitted while such a nonce exists. They are placed into an in-memory retry queue, after which the submission handler returns.

```
indexer/src/operator/sender/transaction.rs
```

```

1  pub async fn handle_transaction_submission(
2      state: &mut SenderState,
3      tx_builder: TransactionBuilder,
4      storage_tx: &mpsc::Sender<TransactionStatusUpdate>,
5  ) {
6      [...]
7      // Park the withdrawal if that tree still has an unresolved ambiguous nonce.
8      // Building its proof now could use a local SMT that disagrees with chain;
9      // the tick drain retries it once process_pending_remints resolves the nonce.
10     if release_tree_index.is_some_and(|tree|
11         state.has_unresolved_ambiguous_nonce(tree)) {
12         // The if-let always matches here: only ReleaseFunds sets release_tree_index.
13         if let TransactionBuilder::ReleaseFunds(builder_with_nonce) = tx_builder {
14             debug!(
15                 nonce = builder_with_nonce.nonce,
16                 "Parking withdrawal: ambiguous nonce in same tree unresolved"
17             );
18             // Mark the row Parked so recovery's Processing sweep does not
19             // quarantine it. Best-effort: the in-memory queue still drives it,
20             // and the next heartbeat re-park repairs a write lost here.
21             let id = builder_with_nonce.transaction_id;
22             if let Err(e) = state.storage.try_park_processing(id).await {
23                 warn!(transaction_id = id, "Park status write failed: {e}");
24             }
25             state.ambiguous_retry_queue.push(builder_with_nonce);
26         }
27         // Always return: a blocked withdrawal is parked, not submitted.
28         return;
29     }
30 }
```

Parking withdrawals behind an unresolved nonce in the same SMT tree

The escrow program rejects a release when the recipient associated token account is absent, which can cause the initiating withdrawal to fail. Once the failed nonce is removed and a pending

SMT rotation advances the tree in `take_pending_rotation_if_ready`, the parked withdrawals are retried with a nonce belonging to the prior tree. The proof builder reports a tree-index mismatch for any unequal expected and current indices, while the error router places every such `ReleaseFunds` builder into `rotation_retry_queue`. The periodic sender loop drains that queue by resubmitting each item, including entries that are requeued during the same drain when the tree index can no longer return to the expected value. This can prevent the drain from completing and halt processing of further withdrawals.

Remediation

In the `TreeIndexMismatch` handling path, distinguish withdrawals awaiting a future rotation from those whose tree has already passed. Retry only when `expected_tree_index` is greater than `current_tree_index`. When `expected_tree_index` is less than `current_tree_index`, treat the withdrawal as terminal and invoke the existing permanent-failure path with its transaction context, ensuring SMT state and remint or failure status are handled instead of requeuing the stale builder.

Patch

Resolved in [PR#184](#).

InitializeMint2 Missing From Admin Instruction Map

ID	Severity	Status
OS-SPF-ADV-03	● Medium	✓ Resolved

Description

The SPL Token `InitializeMint` instruction is designated as admin-only in `ADMIN_INSTRUCTIONS_MAP`, as shown in the mapping below. However, the map does not include `InitializeMint2`, whose discriminator is 20.

```
core/src/transactions.rs
5  const SPL_INITIALIZE_MINT: u8 = 0;
6
7  /// A lazy-initialized static mapping from program_id (Pubkey) to a HashSet of admin
   instruction types (u8)
8  pub static ADMIN_INSTRUCTIONS_MAP: LazyLock<HashMap<Pubkey, HashSet<u8>>> =
9      LazyLock::new(|| HashMap::from([(spl_token::id(),
   HashSet::from([SPL_INITIALIZE_MINT]))]));
10
11 /// Checks if an instruction is an admin-only instruction
12 pub fn is_admin_instruction(program_id: &Pubkey, instruction_type: u8) -> bool {
```

`is_admin_instruction` determines admin status by checking whether the program ID and first instruction-data byte appear in this map. Consequently, `InitializeMint2` instruction is not recognized as admin-only and is treated as a non-admin instruction by the transaction-classification flow. This leaves mint creation through `InitializeMint2` available in the channel without the intended admin-only classification.

Remediation

Add the SPL Token `InitializeMint2` discriminator (20) to the SPL Token entry in `ADMIN_INSTRUCTIONS_MAP` so it receives the same admin-only treatment as `InitializeMint`.

Patch

Resolved in [315a646](#).

Excess Token Transfers Block Reconciliation

ID	Severity	Status
OS-SPF-ADV-04	● Medium	✓ Resolved

Description

Startup reconciliation compares the database-derived expected balance with the on-chain custody balance for each mint and treats any difference as a mismatch. As shown in `compute_mismatch`, an on-chain balance greater than the expected balance increases the mismatch by the same amount.

```
indexer/src/indexer/reconciliation.rs
```

```

212 | pub fn compute_mismatch(db_expected: u64, on_chain_actual: u64) -> u64 {
213 |     on_chain_actual.abs_diff(db_expected)
214 | }
```

Symmetric mismatch calculation

The reconciliation sweep aggregates token accounts owned by the escrow instance, and startup fails when a mismatch exceeds `mismatch_threshold_raw`. Consequently, an attacker who sends extra tokens to a token account belonging to the instance could cause the reconciliation mismatch to exceed the configured threshold and prevent the escrow indexer from starting.

Remediation

Change reconciliation to treat only an on-chain shortfall relative to the database-derived expected balance as a startup-blocking mismatch.

Patch

Resolved in [9cba671](#).

Insufficient Finality for Operator Transactions

ID	Severity	Status
OS-SPF-ADV-05	● Medium	✓ Resolved

Description

The `send_and_confirm` transaction path selects `CommitmentConfig::confirmed()` rather than `finalized` commitment when confirming a sent transaction, as shown in the referenced confirmation logic. Using `confirmed` commitment allows the operator to treat confirmation at a commitment level lower than finalized. This may cause off-chain handling to proceed before the transaction reaches finalized commitment.

Remediation

Use `CommitmentConfig::finalized()` in `send_and_confirm` so transaction handling proceeds only after finalized commitment is reached.

Patch

Resolved in [PR#189](#). The only exception is withdrawal release, which remains on confirmed and is intentionally deferred until Alpenglow due to the complexity and liveness impact of asynchronous finalization.

JIT Mint Retry Bypasses Signature Persistence

ID	Severity	Status
OS-SPF-ADV-06	● Medium	✓ Resolved

Description

`route_poll_results` forwards confirmed transaction failures to `handle_confirmation_result`, including failures classified as `MintNotInitialized`. For a `MintNotInitialized` result, `handle_confirmation_result` verifies that the context has a transaction ID and a cached mint builder, runs just-in-time mint initialization, and directly reissues the resulting mint instruction through `send_and_confirm`, as shown in the JIT retry path.

```
indexer/src/operator/sender/transaction.rs
```

```
1  #[allow(clippy::too_many_arguments)]
2  pub(super) fn handle_confirmation_result<'a>([...]) -> std::pin::Pin<Box<dyn
   std::future::Future<Output = ()> + Send + 'a>> {
3      [...]
4      JitOutcome::Retry(new_instruction) => {
5          info!("JIT verdict: Retry - re-issuing mint instruction");
6          send_and_confirm(
7              state,
8              new_instruction,
9              compute_unit_price,
10             ctx,
11             retry_policy,
12             extra_error_checks_policy,
13             storage_tx,
14         )
15         .await;
16     }[...]
17 }
18 [...]
19 }
```

The JIT path reissues the mint through `send_and_confirm`.

`send_and_confirm` only persists a pre-broadcast signature when both `withdrawal_nonce` and `transaction_id` are present. Therefore, a retried mint with no withdrawal nonce does not receive the signature persistence used by this helper before it is broadcast.

```
indexer/src/operator/sender/transaction.rs
```

```
1  pub(super) async fn send_and_confirm([...]) {
2      [...]
3      if let (Some(_nonce), Some(txid)) = (ctx.withdrawal_nonce, ctx.transaction_id) {
4          if persist_signature_or_abort(
```

```

5         &state.storage,
6         pt,
7         txid,
8         &signature,
9         last_valid_block_height,
10    )
11    .await.is_err() {return;}
12    }
13    [...]
14    }

```

Signature persistence is conditional on a withdrawal nonce.

This diverges from the normal mint submission path, which accepts a `persist` flag and persists the signature before broadcasting when required. Bypassing that path for the JIT retry creates inconsistent recovery handling for the retried mint. If the retry reaches the network but its outcome later becomes uncertain, the operator may lack the durable signature record needed to reconcile it before the transaction is retried or otherwise transitioned, which could permit a duplicate mint.

Remediation

Route a JIT-retried user-fund mint through the same write-ahead persistence and recovery flow as its initial submission. In particular, persist the retry transaction signature, keyed by its transaction ID, before broadcasting it, and preserve the transaction in a recoverable state when its broadcast or confirmation outcome is uncertain.

Patch

Resolved in [PR#186](#).

Database-Loaded Accounts Never Become Evictable

ID	Severity	Status
OS-SPF-ADV-07	● Medium	✓ Resolved

Description

`preload_accounts` loads cache-miss accounts from `AccountsDB` into `BOB`, but initializes their `synced_since` metadata to `None`. This conflicts with the cache-state meaning documented in `bob.rs`, which states `None` represents in-memory state that is newer than the database and must not be evicted. In contrast, these accounts were copied from the database.

core/src/accounts/bob.rs

```

1  pub async fn preload_accounts(&mut self, pubkeys: &[Pubkey]) -> (usize, usize) {
2      [...]
3      // Only fetch the cache-miss keys from the database.
4      let accounts = self.accounts_db.get_accounts(&miss_keys).await;
5      let mut fetched = 0usize;
6      for (index, account_opt) in accounts.iter().enumerate() {
7          if let Some(account) = account_opt {
8              self.accounts.insert(
9                  miss_keys[index],
10                 AccountWithMeta {
11                     account: account.clone(),
12                     synced_since: None,
13                     deleted: false,
14                 },
15             );
16             fetched += 1;
17         }
18     }
19     (fetched, already_cached)
20 }
```

Cache-miss accounts loaded from the database are marked as unsynced.

The garbage collector marks an entry as synced only when it receives matching settlement feedback, and its periodic eviction sweep retains every entry whose `synced_since` is `None`. Consequently, an account that is preloaded and subsequently only read may retain `None` indefinitely and remain in the in-memory cache beyond the configured eviction age. Repeated access to distinct read-only accounts could therefore cause the cache to grow without the intended eviction sweep removing those entries.

Remediation

When inserting an account fetched from `AccountsDB` in `preload_accounts`, initialize `synced_since` to `Some(now)`.

Patch

Resolved in [PR#203](#).

Admin VM Returns Misaligned Loaded Accounts

ID	Severity	Status
OS-SPF-ADV-08	● Medium	✓ Resolved

Description

The admin VM builds the `LoadedTransaction.accounts` vector by appending an entry only when an `InitializeMint` instruction succeeds. The mint public key is resolved from the instruction's account index, but that original index is not retained when the account is appended. In `load_and_execute_sanitized_transactions`, the resulting vector can therefore be shorter than, and ordered independently of, the transaction message's account-key vector.

`BOB::update_accounts` enumerates the returned loaded-account vector and passes that enumeration index to `sanitized_transaction.is_writable(index)`. This assumes each returned account occupies the same position as its corresponding message account key; an admin-VM result that omits preceding message accounts breaks that assumption.

The durable settlement path also processes executed transactions using the loaded accounts and an index-based writable check, as identified in the original finding. Consequently, the inconsistency affects both BOB's in-memory account state and the account updates collected for persistence.

For an admin transaction whose initialized mint is not at the same position as its position in the compacted returned vector, writability may be evaluated against a different message account key. This could allow a returned account to be updated when its actual message entry is read-only, or cause an actual writable account update to be skipped.

Remediation

Make the admin VM preserve the `LoadedTransaction.accounts` contract expected by its consumers by returning account entries in the same order and at the same indices as the transaction message's account keys, including entries needed to retain positional alignment.

Patch

Resolved in [42a4b2e](#).

Failed Transaction Account Updates

ID	Severity	Status
OS-SPF-ADV-09	● Medium	✓ Resolved

Description

`update_accounts` treats every `ProcessedTransaction::Executed` result as eligible for account-cache updates. It iterates the executed transaction's loaded accounts and stores each writable account, including a tombstone when the account has zero lamports and empty data, without checking `executed_transaction.execution_details.status`. An executed transaction whose status indicates failure may therefore update the in-memory writable-account state. Similarly, `settle_transactions` also consumes executed transactions without validating `execution_details.status` before storing account changes.

Remediation

In both `update_accounts` and `settle_transactions`, inspect `executed_transaction.execution_details.status` before storing the account changes.

Patch

Resolved in [PR#177](#).

Zero-Lamport Accounts Retain Data in DB

ID	Severity	Status
OS-SPF-ADV-10	● Medium	✓ Resolved

Description

The settlement stage marks a writable account as deleted only when both its lamport balance is zero and its data is empty. Consequently, an account whose lamports reach zero while its data remains present is treated as an active settlement and is written back to the account database.

The in-memory account view applies the same condition, while the batch database writer deletes only settlements already marked as deleted. This behavior diverges from Agave's deletion behavior and causes zero-lamport accounts with data to persist in the database.

Remediation

Change the deletion predicate for writable settled accounts to mark an account deleted whenever `lamports() == 0`, irrespective of whether its data is empty.

Patch

Resolved in [PR#236](#).

Database Errors Treated as Missing Accounts

ID	Severity	Status
OS-SPF-ADV-11	● Medium	✓ Resolved

Description

`BOB::preload_accounts` loads only cache-miss public keys through `AccountsDB` before account execution. The database lookup returns optional account values, and the preload loop inserts only values that are present. It has no error result to propagate to its caller, as shown in `preload_accounts`.

```
core/src/accounts/bob.rs
1  pub async fn preload_accounts(&mut self, pubkeys: &[Pubkey]) -> (usize, usize) {
2      [...]
3      // Only fetch the cache-miss keys from the database.
4      let accounts = self.accounts_db.get_accounts(&miss_keys).await;
5      let mut fetched = 0usize;
6      for (index, account_opt) in accounts.iter().enumerate() {
7          if let Some(account) = account_opt {
8              self.accounts.insert(
9                  miss_keys[index],
10                 AccountWithMeta {
11                     account: account.clone(),
12                     synced_since: None,
13                     deleted: false,
14                 },
15             );
16             fetched += 1;
17         }
18     }
19     (fetched, already_cached)
20 }
```

The account-loading implementation represents a failed PostgreSQL query or Redis multi-get as a vector of absent accounts, the same representation used for accounts that are not stored. Consequently, a database failure is indistinguishable from a legitimate cache miss at the preload boundary. The execution stage awaits preloading and then continues processing the batch, so it cannot treat such failures as fatal or retry them. This may cause database availability or retrieval failures to be handled as absent account data rather than as storage errors.

Remediation

Change `AccountsDB::get_accounts` and its PostgreSQL and Redis implementations to return a `Result` that distinguishes database and deserialization failures from legitimately absent accounts. Update `BOB::preload_accounts` to propagate that result, and ensure the execution-stage caller stops processing the affected batch or retries the lookup when preloading fails.

Patch

Resolved in [85aea29](#).

Failed Withdrawal Remint Sent to Destination

ID	Severity	Status
OS-SPF-ADV-12	● Medium	✓ Resolved

Description

The withdrawal instruction permits an optional destination address for the Solana-side release. As shown in `WithdrawFundsArgs`, the destination is independent of the withdrawal amount and may be omitted.

```
private-channel-withdraw-program/program/src/processor/withdraw_funds.rs
```

```
72 pub struct WithdrawFundsArgs {
73     pub amount: u64,
74     pub destination: Option<Address>,
75 }
```

Withdrawal destination is optional.

When no destination is supplied, the withdrawal recipient defaults to the user who burned the private-channel tokens. However, when a destination is explicitly supplied, a permanently failed release remints to that destination rather than the user who performed the burn. The burned user's balance may therefore not be restored. This mismatch does not arise when no destination is supplied because the recipient defaults to the withdrawing user.

```
indexer/src/operator/processor.rs
```

```
1  async fn build_release_funds(...) -> Result<TransactionBuilder, OperatorError> {
2      [...]
3      let remint_info = WithdrawalRemintInfo {
4          [...]
5          mint,
6          user: recipient,
7          user_ata: remint_user_ata,
8          [...]
9      };
10     [...]
11 }
```

Remediation

Record the withdrawing signer separately from the release destination when processing withdrawal events.

Patch

Resolved in [61f433b](#).

Panic on Duplicate Transaction Account Keys

ID	Severity	Status
OS-SPF-ADV-13	● Medium	✓ Resolved

Description

The scheduler's `extract_accounts` function obtains a transaction's read and write account locks with `get_account_locks(usize::MAX)` and unconditionally calls `expect` on the result. As shown below, an account-locking error therefore causes a panic rather than allowing the malformed transaction to be rejected.

```
core/src/scheduler/types.rs
43 pub fn extract_accounts(transaction: &SanitizedTransaction) -> (Vec<Pubkey>,
44     Vec<Pubkey>) {
45     // Use Solana's built-in account locking mechanism
46     // We use usize::MAX as the limit to avoid artificial restrictions
47     let account_locks = transaction
48         .get_account_locks(usize::MAX)
49         .expect("Failed to get account locks");
50     let read_accounts = account_locks.readonly.into_iter().copied().collect();
51     let write_accounts = account_locks.writable.into_iter().copied().collect();
52
53     (read_accounts, write_accounts)
54 }
```

Unchecked account-lock construction in `extract_accounts`

The write-node RPC path accepts a serialized transaction, converts it to a sanitized transaction, applies the program allowlist, and queues it for the transaction pipeline. The sequencer uses the DAG scheduler to process queued transactions, which invokes `extract_accounts` while building dependency information. Consequently, a transaction with a repeated public key in its account-key array can reach this panic path when account-lock creation returns `AccountLoadedTwice`, causing denial of service for transaction processing and potentially taking down the node.

Remediation

Make account extraction fallible instead of calling `expect` on `get_account_locks`. Propagate the lock-construction error through the scheduler and sequencer so a transaction whose account keys contain duplicates is rejected or dropped with an error and does not panic the worker.

Patch

Resolved in [16596c1](#).

Overly Broad Withdrawal Pipeline Quarantine

ID	Severity	Status
OS-SPF-ADV-14	● Medium	✓ Resolved

Description

When a withdrawal is classified for quarantine, `process_release_funds` quarantines the offending row and invokes `halt_withdrawal_pipeline`, terminating further withdrawal processing. The rationale documented for this behavior is that a quarantined withdrawal creates a nonce gap that invalidates every later nonce in the same tree. However, `validate_current_tree_index` only verifies that a nonce belongs to the current tree generation, while the release path verifies SMT proofs for individual leaves. Thus, this justification does not establish that later nonces in the same tree must fail.

```
indexer/src/operator/processor.rs
```

```

1  /// Halt the withdrawal pipeline after a poison-pill is detected.
2  ///
3  /// A quarantined withdrawal leaves a permanent nonce gap that the on-chain
4  /// program rejects for every subsequent nonce in the same tree. Rather
5  /// than bleed errors downstream, we stop cleanly:
6  /// 1. Quarantine any rows the fetcher already handed us (drain the rx).
7  /// 2. Flip every other `Pending`/`Processing` withdrawal in the DB to
8  ///    `ManualReview` so the fetcher has nothing left to pull.
9  /// [...]
10 async fn halt_withdrawal_pipeline(
11     storage: &Storage,
12     storage_tx: &mpsc::Sender<TransactionStatusUpdate>,
13     fetcher_rx: &mut mpsc::Receiver<DbTransaction>,
14     poison_id: Option<i64>,
15 ) { [...]}

```

`halt_withdrawal_pipeline` drains withdrawals already received from the fetcher and quarantines them, then invokes the storage sweep for remaining active withdrawals. This converts an error associated with one withdrawal into a pipeline-wide halt and requires the affected rows to receive manual review before processing can resume. The PostgreSQL sweep updates every withdrawal in `pending`, `processing`, or `parked` status to `manual_review`, excluding only the already-quarantined poison row. Since its scope is database-wide for withdrawal transactions, unrelated active withdrawals are also removed from normal processing.

```
indexer/src/storage/postgres/db.rs
```

```

1614 pub async fn quarantine_all_active_withdrawals_internal(
1615     &self,
1616     exclude_id: Option<i64>,
1617 ) -> Result<u64, sqlx::Error> {

```

```

1618     let result = sqlx::query(
1619         r#"
1620         UPDATE transactions
1621         SET status = 'manual_review', updated_at = NOW()
1622         WHERE transaction_type = 'withdrawal'
1623             AND status IN ('pending', 'processing', 'parked')
1624             AND ($1::BIGINT IS NULL OR id <> $1)
1625         "#,
1626     )
1627     .bind(exclude_id)
1628     .execute(&self.pool)
1629     .await?;
1630
1631     Ok(result.rows_affected())
1632 }

```

Database-wide withdrawal status update

Remediation

Change the withdrawal `ErrorDisposition::Quarantine` path to retain the `quarantine_single` update for the offending transaction but continue processing subsequent withdrawals instead of calling `halt_withdrawal_pipeline`, as done in the deposit path in `process_deposit_funds`, which quarantines the individual transaction on a quarantinable processor error and continues its loop. This will avoid unnecessarily placing unrelated active withdrawals into `manual_review` where a nonce gap does not itself prevent independently proven SMT leaves from being processed.

Patch

Resolved in [PR#238](#) and [PR#185](#).

Non-Durable Tree Rotation Dispatch

ID	Severity	Status
OS-SPF-ADV-15	● Medium	✓ Resolved

Description

`process_release_funds` dispatches a `ResetSmtRoot` transaction only while processing a withdrawal whose nonce is a nonzero exact multiple of `MAX_TREE_LEAVES`. The sender's `pending_rotation` field is an in-memory `Option` containing only a `ResetSmtRootBuilder`. And `ResetSmtRoot` carries neither `withdrawal_nonce` nor `transaction_id`. Thus, it does not provide durable recovery state that identifies the boundary withdrawal or records that a rotation remains outstanding after a process restart.

```
indexer/src/operator/processor.rs
```

```

1  pub async fn process_release_funds([...]) -> Result<(), OperatorError> {
2      [...]
3      while let Some(transaction) = fetcher_rx.recv().await {
4          [...]
5          let outcome: Result<(), OperatorError> = async {
6              [...]
7              if let Some(nonce_i64) = transaction.withdrawal_nonce {
8                  let nonce = nonce_i64 as u64;
9                  if nonce > 0 && nonce.is_multiple_of(MAX_TREE_LEAVES as u64) {
10                     let target_tree_index = nonce / MAX_TREE_LEAVES as u64;
11                     [...]
12                 }
13                 [...]
14             }
15             [...]
16         }
17         [...]
18     }

```

Therefore, when reading the current on-chain tree index before dispatching the rotation, on any error from that read, `send_fatal_error` writes nothing on any failure. The error is propagated as a processor error, and `Transient` errors cause the processor task to exit for supervisor restart. Consequently, an RPC failure at this boundary can prevent the sole rotation-dispatch path from running.

During boot recovery, `check_withdrawal` quarantines a withdrawal with no persisted broadcast signatures because it cannot establish whether a release landed. A boundary withdrawal that failed before its rotation or release was broadcast therefore cannot be automatically demoted and replayed through the dispatch path.

indexer/src/operator/recovery.rs

```

1  async fn check_withdrawal(
2      row: &DbTransaction,
3      storage: &Storage,
4      rpc_client: &RpcClientWithRetry,
5  ) -> WithdrawalAction {
6      [...]
7      // No recorded signatures → can't verify a release landed; demoting risks a
8      // double-payout, so page instead.
9      if pending.is_empty() {
10         return WithdrawalAction::Quarantine {
11             reason: "no broadcast signatures recorded; cannot verify release
12                 landed".to_string(),
13         };
14     }
15     [...]
16 }

```

The processor comments describe reprocessing a re-armed boundary row as the means to perform a missed rotation while avoiding a duplicate on-chain rotation. However, the rotation builder lacks withdrawal-specific persistence, and recovery quarantine of the never-broadcast boundary row leaves no durable rotation obligation or status update identifying the required manual re-arm. Later withdrawals can remain blocked when their expected tree state exceeds the on-chain tree state.

indexer/src/operator/processor.rs

```

1  pub async fn process_release_funds([...]) -> Result<(), OperatorError> {
2      [...]
3      while let Some(transaction) = fetcher_rx.recv().await {
4          [...]
5          let outcome: Result<(), OperatorError> = async {
6              [...]
7              // that quarantines below still leaves later withdrawals on the new
8              // tree. Skip if already rotated on-chain: a re-armed boundary row
9              // reprocesses this nonce, and rotating twice skips a tree generation.
10             if let Some(nonce_i64) = transaction.withdrawal_nonce { [...] }
11             [...]
12         }
13         [...]
14     }
15     [...]
16 }

```

Remediation

Persist a tree-rotation obligation before dispatching `ResetSmtRoot`, keyed to the boundary withdrawal nonce and its transaction identifier, with explicit pending, broadcast, confirmed, and failed states.

Patch

Resolved by [PR#232](#), which builds on [PR#209](#) and [PR#185](#).

Backpressure-Induced Duplicate Deposit Mints

ID	Severity	Status
OS-SPF-ADV-16	● Medium	✓ Resolved

Description

`get_and_lock_pending_transactions_internal` selects pending rows, changes their status to `Processing`, commits that status, and returns the rows before the fetcher forwards them through the processor-to-sender pipeline. Consequently, a row can be durably marked as being processed while it is still waiting in an in-memory channel rather than having a persisted mint signature.

```
indexer/src/storage/postgres/db.rs
```

```

1  pub async fn get_and_lock_pending_transactions_internal(
2      &self,
3      transaction_type: TransactionType,
4      limit: i64,
5  ) -> Result<Vec<DbTransaction>, sqlx::Error> {
6      [...]
7      // Update status to Processing in a single query
8      if !transactions.is_empty() {
9          let ids: Vec<i64> = transactions.iter().map(|txn| txn.id).collect();
10         sqlx::query(&format!(
11             "UPDATE transactions SET {} = $1 WHERE {} = ANY($2)",
12             transaction_cols::STATUS,
13             transaction_cols::ID
14         ))
15         .bind(TransactionStatus::Processing)
16         .bind(&ids)
17         .execute(&mut *tx)
18         .await?;
19     }
20     [...]
21 }
```

Pending rows are committed as `Processing` before being returned.

The fetcher forwards each returned row through a guaranteed channel send, while the sender enforces a `MAX_IN_FLIGHT` capacity of 1,000. Under sustained back-pressure, this permits `Processing` deposit rows to remain queued without a durable record that a mint was broadcast. If the queue remains stalled beyond the recovery threshold, recovery can concurrently requeue the database row while the original in-memory copy remains available to resume processing.

As shown in `check_deposit`, recovery treats an empty persisted-signature set as `NotLanded` without querying chain state. The recovery worker maps that outcome to a requeue, allowing the deposit to be fetched again. Because the deposit path stamps `mint_idempotency_memo` but does

not read it before minting, a resumed queued copy and the re-fetched copy may both mint the same deposit.

```

indexer/src/operator/recovery.rs
1  async fn check_deposit(
2      row: &DbTransaction,
3      storage: &Storage,
4      rpc_client: &RpcClientWithRetry,
5  ) -> DepositOutcome {
6      let pending = match load_pending_sigs(storage, row.id).await {
7          Ok(p) => p,
8          Err(reason) => {
9              return DepositOutcome::Ambiguous {
10                  reason: format!("could not verify mint landed ({reason})"),
11              }
12          }
13      };
14
15      if pending.is_empty() {
16          return DepositOutcome::NotLanded;
17      }
18      [...]
19  }

```

An empty persisted-signature set is treated as not landed.

Remediation

Apply back-pressure before transitioning a deposit row to `Processing`.

Patch

Resolved by [PR#197](#), which builds on [PR#155](#).

Cross-Role Pending Remint Recovery

ID	Severity	Status
OS-SPF-ADV-17	● Medium	✓ Resolved

Description

The pending-remint recovery query selects withdrawal rows solely by `pending_remint` status and `withdrawal` transaction type. It does not scope rows to an operator role or channel, allowing operators sharing a database to recover the same pending-remint records.

```

indexer/src/storage/postgres/db.rs
1  pub async fn get_pending_remint_transactions_internal(
2      &self,
3  ) -> Result<Vec<DbTransaction>, sqlx::Error> {
4      sqlx::query_as::(<_, DbTransaction>(&format!(
5          r#"
6          SELECT
7              {}, {}, {}, {}, {}, {}, {}, {}, {}, {},
8              {}, {}, {}, {}, {}, {}, {}, {}, {}, {}, {}, {}
9          FROM transactions
10         WHERE {} = $1 AND {} = $2
11         ORDER BY {} ASC
12         "#, [ ... ]
13     ).bind(TransactionStatus::PendingRemint)
14     ).bind(TransactionType::Withdrawal)
15     ).fetch_all(&self.pool)
16     ).await
17 }

```

Pending-remint recovery query lacks operator or channel scoping.

`recover_pending_remints` restores each selected record, including its stored release signatures, deadline, and `finality_check_attempts`, into the sender's in-memory queue. `process_pending_remints` subsequently classifies those signatures with that sender's `rpc_client`. When a signature has no status entry, classification requires `getBlockHeight`, and an RPC failure produces an `Uncertain` result. The private-channel RPC interface does not expose `getBlockHeight`, so an escrow operator handling a withdrawal operator's recovered record can enter the defer-or-escalate path. Since the attempt counter is persisted and restored, a record already at two attempts reaches the cap on this operator's first classification tick and is marked `ManualReview`.

The status-update guard in `update_transaction_status_internal` permits a `pending_remint` record to be updated to a terminal status, including `ManualReview`. Once that update is written, the legitimate withdrawal operator can no longer perform its terminal status update through this guarded path. This can strand a withdrawal whose private-channel tokens were burned until the record is manually resolved.

indexer/src/storage/postgres/db.rs

```

1  pub async fn update_transaction_status_internal(
2      &self,
3      transaction_id: i64,
4      status: TransactionStatus,
5      counterpart_signature: Option<String>,
6      processed_at: chrono::DateTime<chrono::Utc>,
7  ) -> Result<bool, sqlx::Error> {
8      // Only write non-terminal source states - blocks late writes after recovery.
9      let result = sqlx::query(
10         r#"
11         UPDATE transactions
12         SET
13             status = $2,
14             counterpart_signature = $3,
15             processed_at = $4
16         WHERE id = $1
17             AND status IN ('processing', 'pending_remint')
18         "#,
19     )[..]
20     Ok(result.rows_affected() == 1)
21 }

```

Terminal status update accepts pending-remint rows.

Remediation

Scope pending-remint retrieval and recovery to the operator role and the channel the row belongs to. Persist sufficient ownership information on each transaction, include it in `get_pending_remint_transactions_internal`, and pass the configured operator scope into that query. Also ensure terminal status transitions are performed only by the operator that owns the pending-remint record, so a mismatched operator cannot mark another role's withdrawal as `ManualReview`.

Patch

Resolved in [PR#210](#).

Pre-Broadcast Persistence Failure Leaves Phantom SMT Leaf

ID	Severity	Status
OS-SPF-ADV-18	● Medium	✓ Resolved

Description

`handle_release_funds_transaction` records the release builder, generates an exclusion proof, then inserts the withdrawal nonce into the operator's local SMT and derives the new root before returning the instruction for signing and broadcast. Consequently, the local tree is advanced before `send_and_confirm` has durably recorded the transaction signature or submitted the release.

indexer/src/operator/sender/proof.rs

```

1  pub(super) fn handle_release_funds_transaction(
2      &mut self,
3      builder_with_nonce: Box<ReleaseFundsBuilderWithNonce>,
4      fee_payer: Pubkey,
5      signers: Vec<&'static Signer>,
6      compute_unit_price: Option<u64>,
7      compute_budget: Option<u32>,
8  ) -> Result<InstructionWithSigners, OperatorError> {
9      [...]
10     // Generate exclusion proof BEFORE inserting nonce
11     let exclusion_proof = self.smt_state.generate_exclusion_proof(nonce);
12     // Insert nonce into SMT (updates tree state)
13     if !self.smt_state.insert_nonce(nonce) {
14         return Err(ProgramError::SmtProofFailed {
15             reason: format!("Failed to insert nonce {} (already exists)", nonce),
16         });
17     }
18     .into();
19     // This will be used for inclusion proof
20     let new_root = self.smt_state.current_root();
21     [...]
22 }
```

Local SMT state is advanced while constructing a release instruction.

The write-ahead signature persistence path aborts before broadcast when `insert_release_signature` fails, but returns without invoking the permanent-failure cleanup path. That cleanup normally removes a failed withdrawal nonce from the local SMT. A persistence failure can therefore leave a nonce in the in-memory tree even though its release was never broadcast.

Subsequent release proofs may be generated against this divergent local root rather than the on-chain SMT root, causing on-chain proof validation to reject them with `InvalidSmtProof`. Because the failed pre-broadcast nonce remains in the local state, the operator may be unable to produce valid proofs for later withdrawals until its SMT state is reinitialized.

Remediation

When write-ahead signature persistence fails before broadcast for a release transaction, invoke the same nonce rollback performed by `cleanup_failed_transaction` before returning, while preserving any state required by the recovery workflow.

Patch

Resolved by [PR#204](#), which builds on [PR#149](#).

Startup SMT Validation Ignores Recoverable Withdrawals

ID	Severity	Status
OS-SPF-ADV-19	● Medium	✓ Resolved

Description

The startup SMT reconstruction obtains withdrawal nonces exclusively from withdrawal rows whose status is `completed`. Consequently, a withdrawal whose release has landed on-chain but whose database row remains in `pending_remint` or `manual_review` does not contribute its nonce to the locally rebuilt tree.

```
indexer/src/storage/postgres/db.rs
```

```

1922     pub async fn get_completed_withdrawal_nonces_internal(
1923         &self,
1924         min_nonce: i64,
1925         max_nonce: i64,
1926     ) -> Result<Vec<i64>, sqlx::Error> {
1927         let nonces: Vec<(i64,)> = sqlx::query_as(
1928             r#"
1929             SELECT withdrawal_nonce FROM transactions
1930             WHERE transaction_type = 'withdrawal'
1931               AND status = 'completed'
1932               AND withdrawal_nonce >= $1
1933               AND withdrawal_nonce < $2
1934             ORDER BY withdrawal_nonce ASC
1935             "#,
1936         )
1937         .bind(min_nonce)
1938         .bind(max_nonce)
1939         .fetch_all(&self.pool)
1940         .await?;
1941
1942         Ok(nonces.into_iter().map(|(n,)| n).collect())
1943     }

```

Completed-withdrawal nonce query used for local SMT reconstruction

The boot pre-flight builds an SMT from that incomplete nonce set and compares its root with the on-chain withdrawal root. A differing root returns `SmtRootMismatch`. The operator startup path treats that specific error as a refuse-to-start condition. Thus, an on-chain release represented by a non-completed local row can prevent the sender from starting.

```
indexer/src/operator/sender/state.rs
```

```

1     pub(crate) async fn validate_smt_root(
2         storage: &Storage,

```

```

3     rpc_client: &RpcClientWithRetry,
4     instance_pda: Option<Pubkey>,
5 ) -> Result<SmtState, OperatorError> {
6     [...]
7     if computed_root != onchain_root {
8         error!([...]);
9
10        return Err(crate::error::ProgramError::SmtRootMismatch {
11            local_root: computed_root,
12            onchain_root,
13        })
14        .into();
15    }
16    [...]
17 }

```

SMT reconstruction and root-mismatch failure path

This ordering also prevents the sender's pending-remint recovery from running, even though pending-remint rows persist release signatures for restart handling. Manual-review rows present a further availability concern because there is no code path that transitions them out of that status. The operator may therefore remain unable to start until the database is reconciled with the on-chain withdrawal state.

Remediation

Before treating an SMT-root mismatch as fatal during startup, reconcile withdrawal rows that can represent an already-landed release.

Patch

Resolved in [PR#239](#).

Pre-Broadcast SMT Failures Quarantined as Withdrawals

ID	Severity	Status
OS-SPF-ADV-20	● Medium	✓ Resolved

Description

A withdrawal builder lazily initializes its SMT state before it can construct and sign the release transaction. The `initialize_smt_state` failures caused by transient RPC or PostgreSQL errors are caught by the `Account(_)` | `Storage(_)` arm of `route_builder_error`, which leaves the associated row in `Processing` for recovery. Because this failure occurs before write-ahead signature persistence and broadcast, no release transaction is submitted.

```

indexer/src/operator/sender/transaction.rs
1  pub(super) async fn route_builder_error(
2      state: &mut SenderState,
3      ctx: &TransactionContext,
4      tx_builder: TransactionBuilder,
5      storage_tx: &mpsc::Sender<TransactionStatusUpdate>,
6      err: OperatorError,
7  ) {
8      [...]
9      match err {
10         [...]
11     }
12     e @ OperatorError::Program(ProgramError::SmtRootMismatch { .. })
13     | e @ OperatorError::Program(ProgramError::SmtNotInitialized)
14     | e @ OperatorError::Account(_)
15     | e @ OperatorError::Storage(_) => {
16         [...]
17     }
18 }
19 }

```

Recovery quarantines a withdrawal when no recorded broadcast signatures are available. The recovery dispatcher preserves the distinct withdrawal quarantine outcome, while the deposit path can demote a transaction when its signature check establishes that it was not landed. Consequently, the pre-builder SMT-initialization failure can move a withdrawal that was never broadcast into `ManualReview` instead of requeuing it. Since `ManualReview` is not subsequently selected for processing, the withdrawal cannot be retried automatically while the user's withdrawal-side tokens remain burned until manual intervention.

Remediation

Record and propagate an explicit pre-broadcast outcome for failures that occur before a withdrawal transaction is built, signed, and write-ahead persisted.

Patch

Resolved in [PR#207](#).

Checkpoint Leapfrogging Falsely Proves Release Absence

ID

OS-SPF-ADV-21

Severity

● Medium

Status

✓ Resolved

Description

The refund path treats the escrow indexer's committed checkpoint as proof that an absent `observed_releases` row represents an actual absence. After `release_record` finds no row for a withdrawal nonce, `coverage_verdict` captures a slot bound. It classifies the release as absent solely because the checkpoint has reached that bound. This results in the assumption that every possible release slot has been indexed.

indexer/src/operator/sender/remint.rs

```

1  async fn coverage_verdict(
2      state: &SenderState,
3      entry: &mut PendingRemint,
4      nonce: u64,
5  ) -> ReleaseRecord {
6      [...]
7      match state
8          .storage
9          .get_committed_checkpoint(ProgramType::Escrow.as_label())
10         .await
11     {
12         Ok(Some(checkpoint)) if checkpoint >= bound => ReleaseRecord::ProvenAbsent,
13         Ok(Some(checkpoint)) => ReleaseRecord::Unproven(format!(
14             "no release is on record for nonce {nonce}, but the indexer has only reached
15             slot \
16             {checkpoint} of {bound}, so a release could still be unindexed"
17         )),
18         Ok(None) => ReleaseRecord::Unproven(format!(
19             "no release is on record for nonce {nonce} and the indexer has committed no
20             \
21             checkpoint, so nothing says the record covers slot {bound}"
22         )),
23     }
24 }
```

The `checkpoint >= bound` branch returns `ProvenAbsent`

That assumption is not guaranteed by `CheckpointState::apply`. Contiguous advancement is enforced only while the backfill gate remains open. Outside that condition, the frontier advances to the greatest completed slot. If an `observed_releases` write fails and that slot withholds its checkpoint update, a later slot can still move the high-water mark past it. Because `last_committed_slot` is updated using `GREATEST`, restart resumes after the missed slot, while

`slot_buffers.remove(&slot)` has already discarded the corresponding data, leaving the release permanently absent from storage despite the checkpoint covering a later slot.

```

indexer/src/indexer/checkpoint.rs
75  fn apply(&mut self, slot: u64) -> bool {
76      let before = self.frontier;
77
78      match self.gate {
79          // Still filling a backfill gap – advance only across contiguous slots.
80          Some(target) if self.frontier < target => self.fill_gap(slot, target),
81          // Ungated, or the gap is filled – track the highest slot seen.
82          _ => self.frontier = self.frontier.max(slot),
83      }
84      let advanced = self.frontier > before;
85      // If the frontier moved, mark it so the next flush persists it.
86      self.dirty |= advanced;
87      advanced
88  }

```

The checkpoint frontier uses plain maximum advancement outside the backfill gap

The resulting checkpoint may satisfy the `checkpoint >= bound` branch in `coverage_verdict` even though `observed_releases` does not cover every slot below it. Thus, `coverage_verdict` may return `ProvenAbsent`, creating a false-negative final refund gate and allow tokens to be reminted for a withdrawal whose on-chain release already succeeded. An actual double credit requires a narrow conjunction: the release must have landed, all retained release signatures must be classified as `Dead`, the nonce bitmap must have rotated beyond the nonce, and a retry must have set `release_refused_on_chain`. Nevertheless, the new release-record gate makes the pre-existing non-contiguous checkpoint behavior a security-critical source of false-negative coverage.

Remediation

Require checkpoints to advance only across contiguous slots whose `observed_releases` writes have succeeded, retaining and retrying failed slot buffers. `coverage_verdict` should return `ProvenAbsent` only when this durable contiguous checkpoint covers the release window.

Patch

Resolved in [PR#182](#) and [PR#208](#).

Fallible Boundary Withdrawal Blocks Bitmap Rotation

ID	Severity	Status
OS-SPF-ADV-22	● Medium	✓ Resolved

Description

In `process_release_funds`, the operator calls `build_release_funds` before evaluating whether the withdrawal nonce requires a bitmap rotation. Because the call propagates errors with `?`, malformed public keys, invalid builder data, and other classified errors can exit the per-row processing body before rotation is considered. Deterministic failures are quarantined as `ManualReview`. The fetcher selects only `Pending` rows, so the boundary withdrawal is not automatically retried, and manually re-arming an unchanged row reproduces the failure.

```

indexer/src/operator/processor.rs
1  pub async fn process_release_funds(...) -> Result<(), OperatorError> {
2      [...]
3      while let Some(transaction) = fetcher_rx.recv().await {
4          [...]
5          let outcome: Result<(), OperatorError> = async {
6              [...]
7              // doesn't pay an extra DB/RPC round-trip for `get_mint_metadata`.
8              let release_funds_tx = build_release_funds(processor_state,
9                  &transaction).await?;
10             [...]
11         }
12         [...]
13     }
14 }

```

The boundary predicate that gates initial rotation dispatch accepts only a positive withdrawal nonce that is an exact multiple of `NONCES_PER_GENERATION`, and then calculates the target generation. If that single boundary row fails during the preceding build, execution does not reach the subsequent rotation construction and dispatch established by the original source evidence. Withdrawals belonging to the unopened generation are consequently classified as `NotYetOpen` by the sender and parked while awaiting a rotation that has no remaining automatic trigger, preventing those withdrawals from progressing until an operator intervenes.

```

indexer/src/operator/processor.rs
1  pub async fn process_release_funds(...) -> Result<(), OperatorError> {
2      [...]
3      while let Some(transaction) = fetcher_rx.recv().await {
4          [...]
5          let outcome: Result<(), OperatorError> = async {

```

```

6         [...]
7         if let Some(nonce_i64) = transaction.withdrawal_nonce {
8             let nonce = nonce_i64 as u64;
9             if nonce > 0 && nonce.is_multiple_of(NONCES_PER_GENERATION) {
10                 let target_generation = nonce / NONCES_PER_GENERATION;
11                 let release_funds_state = processor_state
12                     [...]
13             }
14             [...]
15         }
16         [...]
17     }
18     [...]
19 }

```

The stale-row handling in `recover_once` does not resolve the missing rotation. It retrieves stale `Parked` withdrawals and requeues them to `Pending`. After processing resumes against the same on-chain generation, the sender can classify them as `NotYetOpen` and park them again. The requeue path records an informational log and metric but does not escalate repeated parked requeues or emit a new alert, allowing persistent park-requeue churn and a growing withdrawal backlog without a signal tied to the recurring condition.

```

indexer/src/operator/recovery.rs
1  async fn recover_once([...]) -> Result<(), OperatorError> {
2      [...]
3      let stale_parked = storage
4          .get_stale_parked_transactions(threshold, RECOVERY_BATCH_LIMIT, owned_type)
5          .await?;
6      for row in stale_parked {
7          if cancellation_token.is_cancelled() {
8              info!("Recovery sweep cancelled; remaining parked rows deferred");
9              return Ok(());
10         }
11         if !role_owns(program_type, &row) {
12             continue;
13         }
14         requeue_parked(storage, &row, program_type).await;
15     }
16     [...]
17 }

```

Remediation

Detect and dispatch required bitmap rotations before building the boundary withdrawal, making rotation progress independent of row-specific validation failures. Also cap and escalate repeated parked requeues to prevent silent retry loops.

Patch

Resolved in [PR#277](#).

Withdrawal Builder Errors Skip Remint

ID	Severity	Status
OS-SPF-ADV-23	● Low	○ Ack'd

Description

`handle_transaction_submission` routes transaction-building errors to `route_builder_error`. When the generic error arm is reached, it logs the build failure and invokes `send_fatal_error`, transitioning the associated transaction to `Failed`. There are multiple withdrawal-related errors that can reach this path, including `ProgramError::InvalidProof`, `ProgramError::SmtProofFailed`, `ProgramError::InvalidBuilder`, and `OperatorError::MissingBuilder`.

If the builder is for a withdrawal, reaching this generic fatal path marks the transaction `Failed` without a remint. Because a withdrawal burns private-channel tokens before escrowed funds are released, a build-stage failure through this path may leave the withdrawal failed without restoration of the corresponding private-channel balance.

Remediation

Handle withdrawal (`ReleaseFunds`) builder errors separately from generic fatal errors.

Patch

Solana Foundation acknowledged this issue.

Unbounded Recursive ResetSmtRoot Confirmation Retries

ID	Severity	Status
OS-SPF-ADV-24	● Low	○ Ack'd

Description

`send_and_confirm` applies its idempotent retry limit only when `TransactionContext.withdrawal_nonce` is present. `ResetSmtRoot` has no withdrawal nonce, so this limit is not applied to that operation. `ResetSmtRoot` is configured for idempotent retries and is submitted through the blocking `send_and_confirm` path. When confirmation returns `Retry`, the idempotent branch recursively invokes `send_and_confirm`. For a nonce-less reset, each invocation bypasses the retry cap.

```
indexer/src/operator/sender/transaction.rs
```

```

1  pub(super) fn handle_confirmation_result<'a>([...]) -> std::pin::Pin<Box<dyn
    std::future::Future<Output = ()> + Send + 'a>> {
2      [...]
3      RetryPolicy::Idempotent => {
4          metrics::OPERATOR_TRANSACTION_ERRORS
5              .with_label_values(&[pt, "confirmation_timeout"])
6              .inc();
7          warn!("Confirmation failed for idempotent operation - retrying (nonce
            protects against duplicates)");
8          send_and_confirm(
9              state,
10             instruction,
11             compute_unit_price,
12             ctx,
13             retry_policy,
14             extra_error_checks_policy,
15             storage_tx,
16         )
17         .await;
18     }
19     [...]
20 }
```

Under sustained non-confirmation, the reset may therefore continue retrying without a bound while retaining the recursive call chain. Because this operation uses the blocking sender path, this can delay other sender work, including channel-side remints and queue draining, and may eventually exhaust process resources.

Remediation

Apply a retry limit to every idempotent operation, including nonce-less `ResetSmtRoot`.

Patch

Solana Foundation acknowledged this issue.

Missing Source RPC Requirement for Escrow Operator

ID	Severity	Status
OS-SPF-ADV-25	● Low	○ Ack'd

Description

The escrow operator permits `source_rpc_url` to be omitted despite using a source-chain RPC client for operations that require Solana RPC support. When no source client is configured, affected paths silently substitute `rpc_client`, which represents the private-channel endpoint.

```

indexer/src/operator/operator.rs
1  pub async fn run(...) -> Result<(), OperatorError> {
2      [...]
3      // Withdraw operators don't maintain escrow ATA balances, so reconciliation is
        skipped.
4      let reconciliation_handle = if common_config.program_type ==
        crate::config::ProgramType::Escrow
5      {
6          if let Some(reconciliation_escrow) = common_config.escrow_instance_id {
7              let reconciliation_storage = storage.clone();
8              let reconciliation_config = config.clone();
9              let reconciliation_rpc = source_rpc_client
10                 .clone()
11                 .unwrap_or_else(|| rpc_client.clone());
12              let reconciliation_token = cancellation_token.clone();
13              [...]
14          }[...]
15      }[...]
16  }

```

The reconciliation task applies this fallback before periodically comparing escrow ATA balances with database liabilities. When `rpc_url` targets the private channel and `source_rpc_url` is absent, those reconciliation queries are therefore directed to the private-channel RPC endpoint.

```

indexer/src/operator/operator.rs
1  pub async fn run(...) -> Result<(), OperatorError> {
2      [...]
3      // Monitors SOL balance of the feepayer wallet used for ReleaseFunds transactions.
4      let feepayer_monitor_handle =
5          if common_config.program_type == crate::config::ProgramType::Escrow {
6              let feepayer_config = config.clone();
7              let feepayer_rpc = source_rpc_client
8                 .clone()
9                 .unwrap_or_else(|| rpc_client.clone());

```

```

10         let feepayer_program_type = common_config.program_type;
11         let feepayer_token = cancellation_token.clone();
12         [...]
13     }[...]
14 }[...]
15 }

```

The feepayer monitor likewise falls back before obtaining the fee payer’s SOL balance. The escrow processor uses the same pattern for its mint cache, and sender state uses it for both the mint cache and the source RPC client used for reminds. The processor’s escrow path selects `rpc_client` when no source client is supplied, extending the configuration error to mint-cache reads.

indexer/src/operator/processor.rs

```

1  pub async fn run_processor(
2      fetcher_rx: mpsc::Receiver<DbTransaction>,
3      sender_tx: mpsc::Sender<TransactionBuilder>,
4      storage_tx: mpsc::Sender<TransactionStatusUpdate>,
5      program_type: ProgramType,
6      instance_pda: Option<Pubkey>,
7      storage: Arc<Storage>,
8      rpc_client: Arc<crate::operator::RpcClientWithRetry>,
9      source_rpc_client: Option<Arc<crate::operator::RpcClientWithRetry>>,
10 ) {
11     info!("Starting processor");
12     match program_type {
13         [...]
14         ProgramType::Escrow => {
15             // Use source_rpc_client for mint cache if available, otherwise fall back to
16             // rpc_client
17             let mint_rpc_client = source_rpc_client.unwrap_or_else(||
18                 rpc_client.clone());
19             let mut processor_state = ProcessorState::new_with_storage(storage,
20                 mint_rpc_client);
21             [...]
22         }
23     }
24 }

```

Sender state similarly initializes its mint cache and source RPC client from the fallback, so remind operations can use the private-channel endpoint when `source_rpc_url` is omitted.

indexer/src/operator/sender/state.rs

```

1  pub(super) fn new(
2      config: &PrivateChannelIndexerConfig,
3      operator_commitment: CommitmentLevel,
4      instance_pda: Option<Pubkey>,
5      storage: Arc<Storage>,
6      retry_max_attempts: u32,
7      confirmation_poll_interval_ms: u64,
8      source_rpc_client: Option<Arc<RpcClientWithRetry>>,
9  ) -> Result<Self, OperatorError> {
10     // Initialize global RPC client with retry
11     let rpc_client = Arc::new(RpcClientWithRetry::with_retry_config(
12         config.rpc_url.clone(),
13         RetryConfig::default(),
14         CommitmentConfig {
15             commitment: operator_commitment,
16         },
17     ));
18
19     let mint_rpc_client = source_rpc_client.unwrap_or_else(|| rpc_client.clone());
20     let mint_cache = MintCache::with_rpc(storage.clone(), mint_rpc_client.clone());
21     [...]
22 }

```

By contrast, the withdraw operator fails closed during startup when `source_rpc_url` is absent. No equivalent escrow-specific validation is present before the escrow operator starts these tasks.

indexer/src/operator/operator.rs

```

1  pub async fn run([...]) -> Result<(), OperatorError> {
2      [...]
3      if common_config.program_type == crate::config::ProgramType::Withdraw
4          && common_config.source_rpc_url.is_none()
5      {
6          return Err(OperatorError::RpcError(
7              "source_rpc_url required for Withdraw operator: remints must target the
7              source \
8              PrivateChannel, not the Solana destination"
9              .to_string(),
10          ));
11      }
12      [...]
13 }

```

RPC error `-32601` is classified as permanent. Because the private-channel endpoint does not implement `getTokenAccountsByOwner` or `getBalance`, the affected reconciliation and feepayer-monitor calls fail on scheduled iterations without retry, preventing the periodic reconciliation check from continuously detecting discrepancies between escrow balances and recorded liabilities.

```
indexer/src/operator/utis/rpc_util.rs
46 fn is_permanent_rpc_error(e: &client_error::Error) -> bool {
47     let ErrorKind::RpcError(rpc_err) = e.kind() else {
48         return false;
49     };
50     match rpc_err {
51         // Method not supported by this RPC endpoint – protocol-level rejection.
52         RpcError::RpcResponseError { code: -32601, .. } => true,
53         // "AccountNotFound" is a definitive answer, not a transient failure.
54         RpcError::ForUser(msg) => msg.contains("AccountNotFound"),
55         _ => false,
56     }
57 }
```

Remediation

Add an escrow-operator startup validation equivalent to the existing withdraw-operator check that returns an error when `program_type` is `Escrow` and `source_rpc_url` is absent. Then ensure the source-chain operations currently using `unwrap_or_else(|| rpc_client.clone())` receive the validated source RPC client directly, preventing fallback to the private-channel endpoint.

Patch

Solana Foundation acknowledged this issue.

Fee Payer Monitor Tracks Operator Wallet

ID	Severity	Status
OS-SPF-ADV-26	● Low	○ Ack'd

Description

`run_feepayer_monitor` identifies the monitored wallet with `SignerUtil::get_operator_pubkey()` and repeatedly records, logs, and evaluates that wallet's balance against the low-balance threshold.

```

indexer/src/operator/feepayer_monitor.rs
1  pub async fn run_feepayer_monitor(
2      config: OperatorConfig,
3      rpc_client: Arc<RpcClientWithRetry>,
4      program_type: ProgramType,
5      cancellation_token: CancellationToken,
6  ) -> Result<(), OperatorError> {
7      let feepayer_pubkey = SignerUtil::get_operator_pubkey();
8      let label = program_type.as_label();
9      [...]
10 }

```

The transaction builders place `admin_signer` first for `ReleaseFunds` and `ResetSmtRoot`, while `InitializeMint` and `Mint` contain only `admin_signer`.

```

indexer/src/operator/utis/instruction_util.rs
112 pub fn signers(&self) -> Vec<&'static Signer> {
113     match self {
114         Self::ReleaseFunds(_) | Self::ResetSmtRoot(_) => {
115             vec![SignerUtil::admin_signer(), SignerUtil::operator_signer()]
116         }
117         Self::InitializeMint(_) | Self::Mint(_) => vec!
118             [SignerUtil::admin_signer()],
119     }
120 }

```

`handle_transaction_builder` derives the transaction fee payer from the first signer. Consequently, the transactions produced by these builders use the admin key as their fee payer, rather than the operator key monitored by `run_feepayer_monitor`.

indexer/src/operator/sender/transaction.rs

```

1  pub(super) async fn handle_transaction_builder(
2      &mut self,
3      tx_builder: TransactionBuilder,
4  ) -> Result<InstructionWithSigners, OperatorError> {
5      let signers = tx_builder.signers();
6      let compute_unit_price = tx_builder.compute_unit_price();
7      let compute_budget = tx_builder.compute_budget();
8      // For now fee payer is always the first signer
9      let fee_payer = match signers.first() {
10         Some(s) => s.pubkey(),
11         None => {
12             return Err(ProgramError::InvalidBuilder {
13                 reason: "No signers provided".to_string(),
14             }
15             .into())
16         }
17     };
18     [...]
19 }

```

This mismatch leaves the balance that pays transaction fees without the monitor's metric and low-balance warning coverage. The admin wallet may therefore become depleted without the configured monitor reporting its low balance, which may prevent the operator from submitting transactions until it is replenished.

Remediation

Change `run_feepayer_monitor` to query the same fee-payer public key selected by transaction construction.

Patch

Solana Foundation acknowledged this issue.

Nonce-Keyed Cleanup Removes Unrelated Mint Builder

ID	Severity	Status
OS-SPF-ADV-27	● Low	○ Ack'd

Description

SenderState caches each MintToBuilder in mint_builders under the mint transaction's txn_id, establishing transaction ID as the cache key.

```

indexer/src/operator/sender/transaction.rs
1  pub(super) async fn handle_transaction_builder( &mut self, tx_builder:
    TransactionBuilder,) -> Result<InstructionWithSigners, OperatorError> {
2      [...]
3      match tx_builder {[...]}{
4          TransactionBuilder::Mint(ref builder_with_txn_id) => {
5              // Cache the builder for potential JIT retry
6              self.mint_builders.insert(
7                  builder_with_txn_id.txn_id,
8                  builder_with_txn_id.builder.clone(),
9              );[...]
10         }[...]
11     }
12 }

```

Mint builders are cached by transaction ID.

When a mint requires just-in-time initialization, try_jit_mint_initialization retrieves the cached builder using its supplied transaction_id. If no entry exists, it returns JitOutcome::PermanentFailure.

```

indexer/src/operator/sender/mint.rs
1  pub(super) async fn try_jit_mint_initialization(
2      state: &mut SenderState,
3      transaction_id: i64,
4      instruction: InstructionWithSigners,
5  ) -> JitOutcome {
6      // 1. Get cached builder + extract mint.
7      let Some(builder) = state.mint_builders.get(&transaction_id).cloned() else {
8          return JitOutcome::PermanentFailure(format!(
9              "no cached MintToBuilder for transaction_id {}",
10             transaction_id
11         ));
12     };[...]
13 }

```

JIT initialization looks up the cached builder by transaction ID and permanently fails when it is absent.

The successful mint path in `handle_success` removes the cached builder using `transaction_id`, consistent with the insertion and lookup key. In contrast, `cleanup_failed_transaction` passes a withdrawal nonce converted to `i64` to `cleanup_mint_builder`. This supplies a nonce-derived value to a cache whose mint builders are keyed by transaction ID.

Consequently, if a failed withdrawal nonce equals an outstanding mint transaction ID, failed-withdrawal cleanup can remove that unrelated mint's builder. A later JIT initialization for that mint then finds no cached builder and is routed to permanent failure in `try_jit_mint_initialization`.

Remediation

Do not invoke `cleanup_mint_builder` from withdrawal-nonce cleanup using a converted nonce. Restrict removal from `mint_builders` to the associated mint transaction ID, passed explicitly from mint-specific terminal paths. Keep withdrawal cleanup limited to nonce-keyed withdrawal state, and retain transaction-ID-based removal for mint completion or failure.

Patch

Solana Foundation acknowledged this issue.

Token-2022 Accounts Abort Escrow Reconciliation

ID	Severity	Status
OS-SPF-ADV-28	● Low	○ Ack'd

Description

`fetch_escrow_balances_by_mint` queries token accounts for both the SPL Token and Token-2022 programs. In its binary-data branch, it decodes each returned account with the legacy `spl_token::state::Account::unpack` routine, as shown below.

```

indexer/src/operator/escrow_sweep.rs
1  pub async fn fetch_escrow_balances_by_mint(
2      rpc_client: &RpcClientWithRetry,
3      escrow_instance_id: Pubkey,
4  ) -> Result<HashMap<Pubkey, u64>, EscrowSweepError> {
5      [...]
6      let (mint, amount) = if let Some(decoded) =
7          keyed_account.account.data.decode() {
8          let token_account =
9              TokenAccount::unpack(&decoded).map_err(|e| EscrowSweepError {
10                  reason: format!(
11                      "Failed to parse token account for program {token_program_id}:
12                      {e}"
13                  ),
14              })?;
15              (token_account.mint, token_account.amount)
16          }
17      }
18      [...]
19  }

```

Legacy binary account decoding in the escrow sweep

A Token-2022 associated token account carrying the `ImmutableOwner` extension exceeds the fixed 165-byte layout required by that unpacking routine. The resulting error is propagated with `?`, so one such account causes the complete escrow balance sweep to fail rather than allowing the remaining accounts to be processed.

The startup path calls `run_startup_reconciliation` for an escrow instance before continuing indexer initialization, and propagates any reconciliation error, as shown in the below snippet. Consequently, an escrow indexer encountering the affected Token-2022 account fails to start. This may halt deposit ingestion and the corresponding minting workflow while the separate withdrawal path remains available to release escrowed funds.

indexer/src/indexer/indexer.rs

```

1  pub async fn run([...]) -> Result<(), IndexerError> {
2      [...]
3      if !backfill_only {
4          match (common_config.program_type, common_config.escrow_instance_id) {
5              (ProgramType::Escrow, Some(seed)) => {
6                  run_startup_reconciliation(
7                      &indexer_config.reconciliation,
8                      common_config.program_type,
9                      &storage,
10                     &common_config.rpc_url,
11                     &seed,
12                 )
13                 .await?;
14             }[...]
15         }
16     }
17     [...]
18 }

```

Startup reconciliation invocation and error propagation

Remediation

Parse binary accounts returned for the Token-2022 program with a Token-2022 extension-aware account decoder that supports layouts containing extensions such as `ImmutableOwner`, while retaining the legacy decoder for SPL Token accounts.

Patch

Solana Foundation acknowledged this issue.

Unscoped CPI Error Decoding Can Stall Bitmap Rotations

ID	Severity	Status
OS-SPF-ADV-29	● Low	○ Ack'd

Description

`parse_program_error` interprets every transaction-level custom error with a value from 11 through 14 as a withdrawal-bitmap error. It discards the failing instruction index and performs no check that the error originated from the escrow program, so identical error values from another program are assigned escrow-specific semantics.

indexer/src/operator/utis/transaction_util.rs

```

216 pub fn parse_program_error(
217     err: &solana_sdk::transaction::TransactionError,
218 ) -> Option<PrivateChannelEscrowProgramError> {
219     match err {
220         solana_sdk::transaction::TransactionError::InstructionError(
221             -,
222             InstructionError::Custom(code),
223         ) => {
224             match *code {
225                 11 =>
226                     Some(PrivateChannelEscrowProgramError::InvalidWithdrawalBitmap),
227                 12 => Some(PrivateChannelEscrowProgramError::NonceAlreadyUsed),
228                 13 =>
229                     Some(PrivateChannelEscrowProgramError::NonceOutsideCurrentGeneration),
230                 14 => Some(PrivateChannelEscrowProgramError::UnexpectedGeneration),
231                 _ => None, // Ignore other program errors
232             }
233         }
234     }
235     _ => None,
236 }
```

Custom error codes 11-14 are decoded without failed-program provenance

`process_release_funds` invokes the Token or Token-2022 transfer using `TransferChecked2022` and propagates an invocation failure with `?`. Consequently, a Token-2022 CPI error can escape the release instruction with its original custom value. In particular, Token-2022 Custom(14) would be decoded by the indexer as `UnexpectedGeneration` rather than as a token-program failure.

private-channel-escrow-program/program/src/processor/release_funds.rs

```

1 pub fn process_release_funds([...]) -> ProgramResult {
2     [...]
3     TransferChecked2022 {
4         from: instance_ata_info,
```

```

5         to: user_ata_info,
6         authority: instance_info,
7         amount: args.amount,
8         token_program: token_program_info.address(),
9         mint: mint_info,
10        decimals: get_mint_decimals(mint_info)?,
11    }
12    .invoke_signed(&[signer])?;
13    [...]
14 }

```

`handle_confirmation_result` treats `UnexpectedGeneration` as evidence that a bitmap rotation already landed, without checking that the transaction is a `RotateBitmap` operation. For a `ReleaseFunds` transaction, `clear_rotation_retry_state` is a no-op, and this branch neither updates the withdrawal row nor removes its nonce from `in_flight_withdrawals`. A retained in-flight withdrawal prevents `take_pending_rotation_if_ready` from dispatching a pending rotation, so a misclassified release failure may stall subsequent bitmap rotations indefinitely within the running operator. The specific transfer-hook route is currently unreachable because `validate_token2022_extensions` rejects Token-2022 mints carrying a transfer hook, but the error decoder and confirmation handler do not themselves enforce that assumption.

Remediation

Decode custom errors only after verifying the originating program and expected transaction type, preserving the failed instruction context. Restrict `UnexpectedGeneration` handling to `RotateBitmap`, and route release/CPI failures through normal withdrawal cleanup and retry logic.

Patch

Solana Foundation acknowledged this issue.

5. Suggestions

Here, we present a discussion of the 3 suggestions we identified during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Severity	Status	Description
OS-SPF-SUG-00	● Informational (Core)	✗ To-do	Single-Byte Admin Instruction Identifiers
OS-SPF-SUG-01	● Informational (Indexer)	✗ To-do	Fetcher Poll Sleep Delays Shutdown
OS-SPF-SUG-02	● Informational (Indexer)	✗ To-do	Permanent Suppression of Orphan Deposit Alerts

Single-Byte Admin Instruction Identifiers

ID	Severity	Status
OS-SPF-SUG-00	● Informational	✗ To-do

Description

`ADMIN_INSTRUCTIONS_MAP` associates each program with admin instruction identifiers, while `is_admin_instruction` uses those identifiers to classify instructions. As shown in the mapping declaration, the identifiers are represented as `u8`, limiting an entry to a single byte.

core/src/transactions.rs

```

1  /// A lazy-initialized static mapping from program_id (Pubkey) to a HashSet of admin
   instruction types (u8)
2  pub static ADMIN_INSTRUCTIONS_MAP: LazyLock<HashMap<Pubkey, HashSet<u8>>> =
3  LazyLock::new(|| HashMap::from([(spl_token::id(),
   HashSet::from([SPL_INITIALIZE_MINT]))]));

```

Admin instruction identifiers are stored as single-byte values.

The transaction classification and execution routing paths both derive the instruction type from the first byte of instruction data before consulting this map. This representation cannot identify an eight-byte Anchor instruction discriminator in full, which would prevent the map from supporting Anchor-program admin instructions if such support is added in the future.

Remediation

Change `ADMIN_INSTRUCTIONS_MAP` and `is_admin_instruction` to store and compare complete instruction-discriminator byte sequences rather than a `u8`.

Fetcher Poll Sleep Delays Shutdown

ID	Severity	Status
OS-SPF-SUG-01	● Informational	✗ To-do

Description

`run_fetcher` checks `cancellation_token` only at the start of each polling-loop iteration. After it fetches and forwards pending transactions, the function waits for the configured database polling interval before it can evaluate that check again.

```
indexer/src/operator/fetcher.rs
```

```
1 pub async fn run_fetcher([...]) -> Result<(), OperatorError> {
2     [...]
3     loop {
4         [...]
5         // Sleep between polls
6         tokio::time::sleep(config.db_poll_interval).await;
7     }
8     [...]
9 }
```

The poll delay in `run_fetcher` is awaited directly. Consequently, a cancellation received while this wait is pending is not observed until the sleep completes, which can delay the fetcher's return by up to `db_poll_interval`.

```
indexer/src/operator/fetcher.rs
```

```
1 pub async fn run_fetcher([...]) -> Result<(), OperatorError> {
2     [...]
3     loop {
4         // Check for cancellation
5         if cancellation_token.is_cancelled() {
6             info!("Fetcher received cancellation signal, stopping...");
7             break;
8         }
9         match storage.count_pending_transactions(transaction_type).await { [...] }
10        [...]
11    }
12    [...]
13 }
```

The feepayer monitor demonstrates an interruptible wait pattern, where it races its interval sleep against cancellation and exits immediately when cancellation wins. The recovery worker similarly selects between cancellation and its interval tick.

indexer/src/operator/feepayer_monitor.rs

```

1  pub async fn run_feepayer_monitor(
2      config: OperatorConfig,
3      rpc_client: Arc<RpcClientWithRetry>,
4      program_type: ProgramType,
5      cancellation_token: CancellationTokens,
6  ) -> Result<(), OperatorError> {
7      [...]
8      loop {
9          [...]
10         tokio::select! {
11             _ = tokio::time::sleep(config.feepayer_monitor_interval) => {},
12             _ = cancellation_token.cancelled() => {
13                 info!("Feepayer monitor received cancellation signal during sleep,
14                     stopping...");
15                 break;
16             }
17         }
18     }
19     info!("Feepayer monitor stopped gracefully");
20     Ok(())
21 }

```

The feepayer monitor races its wait against cancellation.

This delay gates the operator shutdown pipeline. The operator moves `processor_tx` into the fetcher task, and shutdown waits for the fetcher before draining the processor. Therefore, the processor channel cannot close until the fetcher exits and drops its sender. Since the sender drain follows processor drain, a fetcher waiting in its poll sleep can delay downstream shutdown stages.

Remediation

Replace the unconditional poll sleep in `run_fetcher` with a `tokio::select!` that waits for either `tokio::time::sleep(config.db_poll_interval)` or `cancellation_token.cancelled()`.

Permanent Suppression of Orphan Deposit Alerts

ID	Severity	Status
OS-SPF-SUG-02	● Informational	✗ To-do

Description

The reconciliation service retains orphan deposit IDs in `previously_alerted_orphans` for the lifetime of `run_reconciliation`. On its baseline pass, `check_orphan_deposit_rows` records every successfully alerted orphan ID in this set, as shown in the below excerpt.

```

indexer/src/operator/reconciliation.rs
1  async fn check_orphan_deposit_rows(
2      storage: &Arc<Storage>,
3      previously_alerted_orphans: &mut Option<HashSet<i64>>,
4      webhook_client: &WebhookClient,
5      webhook_url: &Option<String>,
6  ) {
7      [...]
8      match previously_alerted_orphans {
9          None => {
10             [...]
11             match send_orphan_deposit_alert(webhook_url, &orphans,
12                 webhook_client).await {
13                 Ok(()) => {
14                     *previously_alerted_orphans = Some(orphans.into_iter().collect());
15                 }
16                 Err(e) => {
17                     error!("Failed to send orphan deposit webhook alert: {}", e);
18                 }
19             }
20             [...]
21         }
22     }
23 }

```

On subsequent reconciliation ticks, `check_orphan_deposit_rows` filters out every ID already present in the retained set and returns without logging or sending a webhook when no newly observed IDs remain. Thus, a persistent orphan deposit is surfaced only once during a process lifetime. Consequently, orphan deposits that remain unresolved after their initial notification receive no re-notification or escalation, and the routine does not expose an outstanding-orphan count. Operators may therefore lack recurring visibility into unresolved orphan deposit rows until the reconciliation process restarts or an additional orphan row is observed.

Remediation

Track the current outstanding orphan set separately from notification history.

Appendix A.

Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the general findings.

Severity	Description
● Critical	Vulnerabilities that immediately result in a loss of availability, consensus divergence, or exploitable memory corruption with minimal preconditions.
● High	Vulnerabilities that may result in consensus divergence or denial of service but require specific transaction or network conditions to exploit.
● Medium	Vulnerabilities that may result in incorrect behavior or state divergence under specific configurations or edge cases.
● Low	Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.
● Informational	Best practices to mitigate future security risks. These are classified as general findings.

Resolution status is one of  Resolved,  Ack'd, or  To-do.

Appendix B.

Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

The design review asks whether the programs's architecture and operating assumptions support its intended behavior. We study the inputs it accepts, the decisions it makes, the information shared between components, and the services or participants it trusts. We then consider how it behaves when those inputs are unexpected or conflicting. The goal is to find conditions that can produce an incorrect decision, inconsistent state, or loss of service.

Design problems often arise from interactions rather than one defective function. Two components may each process a value correctly but interpret it differently. A failure may also leave one component ready to continue while another treats the same operation as incomplete. Local checks can look correct in both cases, so we follow the complete operation across component boundaries.

The implementation review asks whether the code matches the design. We trace untrusted data from its entry point to the point where it affects behavior, checking the assumptions made along the way. The review covers validation, error handling, state updates, concurrency, persistence, resource use, and cleanup. Its exact focus depends on the target's language, runtime, and architecture rather than a fixed list of bug classes.

The boundary between design and implementation is not exact. Missing validation may be a local coding error, while the same omission across several components may reflect an unclear rule. We classify a finding by its root cause, but judge its impact from the resulting behavior and the conditions needed to trigger it.

We begin by reading the target as a team and agreeing on its main components, trust boundaries, and expected behavior. Auditors then review different paths through the system and compare notes where those paths meet. We reproduce suspected issues with focused tests or traces when the setup allows. Before reporting a finding, another auditor reviews its trigger, impact, and proposed remediation.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process